Week 9 Lab Handout

API Security — Keys, Rate Limits, and What You Accidentally Expose

Time: 110 minutes

Bring: your laptop, your working API and `README.md` from Weeks 5–8, and your ring partner's name (they'll be the first consumer of your key)

Before you start: right now, anyone with your URL can call your API

> Last week you published docs that tell a stranger exactly how to use your API. That was the goal — and it has a side effect. Everything that makes your API easy to use for your ring partner makes it equally easy to use for anyone else who finds the URL: to read every record, to hammer it a thousand times a second, or to take it down by accident with a buggy loop.

Security this week is not about making your API impossible to attack. It's three concrete, achievable things: **know who is calling** (authentication), **limit how much any one caller can do** (rate limiting), and **check what your responses give away** (data exposure). Do those three well and you've closed the doors most real student and startup APIs leave open.

Part A — Plan Your Keys (10 min)

1. List every consumer of your API. For now that is your downstream ring partner, plus **your own team** (for testing). Each consumer gets **its own key** — never one shared key for everybody, because then you can't tell callers apart or revoke one without breaking the rest.
2. Decide the rule: **every endpoint requires a valid key**, with no exceptions for "harmless" GETs. (A public health-check endpoint is the only thing you may leave open, and only if you can justify it in Part D.)
3. Decide where keys will live. The answer is **environment variables**, not your source code. A key committed to git is a leaked key — it stays in the history even if you delete the file later.

Part B — Add API Key Authentication (30 min)

The pattern: the caller sends a key in a header on every request; your API checks it before doing anything else; no valid key means a **401**, and the handler never runs.

Here is the pattern in Express (adapt it to your own framework — the idea is identical in Flask, Django, Laravel, and the rest):

```
// Keys come from the environment, never from source code.
const VALID_KEYS = (process.env.API_KEYS || '').split(',').filter(Boolean);

function requireApiKey(req, res, next) {
  const key = req.get('X-API-Key');
  if (!key || !VALID_KEYS.includes(key)) {
    return res.status(401).json({ message: 'Missing or invalid API key' });
  }
  req.apiKey = key;
  next();
}
```

```
app.use(requireApiKey); // must be registered BEFORE your routes
```

Steps:

1. Generate one key per consumer. Make them long and random, for example:

```
node -e "console.log(require('crypto').randomBytes(24).toString('hex'))"
```

2. Put the keys in a `.env` file (or your host's environment settings), and add `.env` to `.gitignore` **before your first commit of it**. Commit a `.env.example` with placeholder values so teammates know what to set.
3. Add the middleware so it runs before every route.
4. Prove it with three requests against your running server:

```
curl -i http://localhost:3000/tasks # expect 401
curl -i -H "X-API-Key: wrong" http://localhost:3000/tasks # expect 401
curl -i -H "X-API-Key: <your-real-key>" http://localhost:3000/tasks # expect 200
```

5. Send your ring partner **their** key privately (direct message, not the class channel and not your repo). Make sure the key reaches the right person only.

> **401 vs 403:** 401 means "I don't know who you are" (missing or bad key). 403 means "I know who you are, and you're not allowed to do this." With one level of access, you'll only use 401 this week — but know the difference, because real APIs with several permission levels use both.

Part C — Add Rate Limiting (25 min)

Authentication tells you who is calling. Rate limiting stops any one caller from calling too much — whether that's abuse or just a consumer's infinite loop.

A simple in-memory limiter, counting requests **per key** in a time window:

```
function rateLimit({ windowMs, max }) {
  const hits = new Map(); // key -> { count, resetAt }
  return (req, res, next) => {
    const id = req.apiKey || req.ip;
    const now = Date.now();
    let entry = hits.get(id);
    if (!entry || now >= entry.resetAt) {
      entry = { count: 0, resetAt: now + windowMs };
      hits.set(id, entry);
    }
    entry.count += 1;
    if (entry.count > max) {
      const retryAfter = Math.ceil((entry.resetAt - now) / 1000);
      res.set('Retry-After', String(retryAfter));
      return res.status(429).json({ message: 'Too many requests', retryAfter });
    }
    next();
  };
}
```

```
app.use(requireApiKey);
app.use(rateLimit({ windowMs: 60000, max: 100 })); // 100 requests per minute
```

Steps:

1. Add rate limiting to **at least your most-called endpoint** (applying it to every route, as above, is fine and simpler).
2. Choose a threshold you can defend. 100 per minute is a reasonable default; write down *why* you chose your number in Part E.
3. **Demonstrate it.** Temporarily lower the limit to something tiny (say 3), then:

```
for i in 1 2 3 4 5; do
  curl -s -o /dev/null -w "%{http_code} " -H "X-API-Key: <your-real-key>"
http://localhost:3000/tasks
done
```

You should see `200 200 200 429 429`. Screenshot or copy this output — it's part of your deliverable. Then set the limit back to your real value.

4. Confirm the 429 response carries a `Retry-After` header and a JSON body, so a well-behaved consumer knows when to try again.

> **Honest limits of this limiter:** it lives in your server's memory, so it resets when you restart, and it won't work correctly if you run several server copies. That's fine for this course. Production systems use a shared store such as Redis.

Part D — Security Self-Audit (25 min)

Open every response your API can return — success responses *and* error responses — and read them as an attacker would. Check each of these:

- **Sensitive fields:** any password hash, token, internal flag, or email address that the consumer has no reason to see?
- **Other people's data:** can a caller read records belonging to someone they shouldn't see, just by changing an ID in the URL?
- **Error messages:** do any errors reveal a stack trace, a file path, a database query, or a table name?
- **Write endpoints:** can a caller send an extra field (like `isAdmin` or `id`) and have your code save it?
- **Keys in the wrong place:** is any key committed in your repo history, printed in your logs, or sitting in your README?

The most common fix is to stop returning raw database rows. Build the response from an explicit list of fields instead:

```
// Leaks everything in the row, including passwordHash
res.json(user);

// Returns only what the contract promises
res.json({ id: user.id, name: user.name });
```

For each item you check, write down what you looked at and what you found — including "nothing wrong" if that's true. Fix anything you find.

Part E — Update Docs, Write the Audit, and Push (20 min)

1. **Update** `README.md`**.** Replace last week's auth placeholder with the real thing: which header to send, what a valid request looks like, what a 401 and a 429 response look like, and your rate limit. **Never put a real key in the README** — use `<your-api-key>` as a placeholder.
2. **Update** `openapi.yaml` to declare the API key security scheme and the 401 and 429 responses, so your rendered docs stay accurate.
3. **Create** `SECURITY_AUDIT.md` at your repo root, covering:

- What you checked (use the Part D checklist)
- What you found, and what you fixed
- Your rate limit threshold and why you chose it

4. Run your Week 7 test suite again. Add at least two new tests: one confirming a request without a key gets 401, and one confirming a valid key gets 200.
5. Commit and push:

```
git add . SECURITY_AUDIT.md README.md openapi.yaml .env.example
git commit -m "Week 9: API key auth, rate limiting, security audit"
git push
```

Before you push, run `git status` and confirm `.env` is **not** listed.

This week's deliverable checklist

- Every endpoint requires a valid API key; a request with no key or a wrong key gets 401
- Each consumer has its own key, stored outside source code; `.env` is not committed
- Rate limiting works, demonstrated with a real 429 (screenshot or terminal output)
- The 429 response includes a `Retry-After` header and a JSON body
- `README.md` and `openapi.yaml` describe the real auth and rate limit, with no real key shown
- `SECURITY_AUDIT.md` documents what was checked, found, and fixed
- New tests cover the 401 and 200 cases, and the whole suite passes
- All code committed and pushed

Rubric (25 pts)

Criterion	Points
Auth correctly implemented and enforced on all endpoints	10
Rate limiting functional and demonstrated	6
Docs updated accurately	4
Security audit shows genuine review	5

Common problems and quick fixes

Problem	Fix
Every request gets 401, even with the right key	Check the header name matches exactly (`X-API-Key`), and that your `.env` is actually loaded — print the number of keys loaded (never the keys themselves) at startup
Some endpoints are still open	The middleware is registered after those routes. Order matters: `app.use(requireApiKey)` must come before your route definitions
The rate limit never triggers	You may be sending requests with different keys, or the window reset between requests. Lower the limit to 3 and send 5 requests quickly
You committed `.env` or a real key by accident	Treat that key as leaked: generate a new one, update your environment, and tell your ring partner. Deleting the file does not remove it from git history
Your partner's integration broke after you added auth	Expected — send them their key and the header name. This is why Part B step 5 matters, and why Week 13's lesson on communicating changes exists
Not sure what counts as sensitive	If the consumer's app never needs the field to do its job, don't return it. When in doubt, leave it out and add it back only if a partner asks

Next week: Integration Week. You'll call your partner's live API — with the key they just gave you — from your own app.